PERFORMANCE

Zerado landing page — Core Web Vitals & performance report

ForgePlay deliverable · engineered artifact

FlowForge
ENGINEERED ARTIFACT

DRAWING TITLE

Zerado landing page — Core Web Vitals & performance report

DISCIPLINE	SHEET	DWG NO.	REV	STATUS
PERFORMANCE	01 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

LEGEND Field — blueprint · Ink — annotation · Accent — registration · Pass — verdict

Rendered from performance-report.md

powered by **FlowForge**

Lane: `performance` (Core Web Vitals). Accessibility, keyboard, cross-browser and content-honesty are owned by the parallel `qa` lane and are deliberately not covered here. **Target:** `http://localhost:4321/` — Astro static build, `site/dist/` **Date:** 2026-08-24 **Tooling:** Lighthouse 13.4.1 (npx) driving Google Chrome 151.0.7922.174 · puppeteer-core 23 + CDP for the browser forensics **Raw artifacts:** `performance/raw/` (6 Lighthouse JSON runs, 2 median HTML reports, `browser-forensics.json`, and the three measurement scripts)

FlowForge

DRAWING TITLE

Zerado landing page — Core Web Vitals & performa... report

SHEET	DWG NO.	REV	STATUS
02 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

§01 Verdict: PASS

METRIC	THRESHOLD	MOBILE (MEDIAN)	DESKTOP (MEDIAN)	RESULT
LCP	≤ 2.5 s	**1.509 s**	**0.368 s**	PASS
CLS	≤ 0.1	**0.000**	**0.000**	PASS
TBT (INP proxy)	≤ 200 ms	**0 ms**	**0 ms**	PASS
Performance score	—	**100**	**100**	PASS

All three runs per form factor scored 100, with under 10 ms of spread between runs — this is a stable result, not a lucky sample.

The honest caveat. These numbers come from a localhost preview server, and there are two distinct things to separate:

1. **Network throttling *was* applied.** Lighthouse ran with `throttlingMethod: "simulate"` — mobile at 150 ms RTT / 1638.4 kbps / 4× CPU slowdown, desktop at 40 ms RTT / 10240 kbps / 1× CPU. So the mobile figures are *not* raw localhost speeds; they are modelled slow-4G.
2. **Server latency was not.** The origin responded in **2 ms** (measured; `server-response-time` audit). A real origin or CDN edge will not. Lighthouse's simulation adds its own per-request latency on top of the *observed* server time, so whatever TTFB production actually has lands on top of these numbers. See "What would change on a real deployment".

The margin is large enough that this caveat does not threaten the verdict: mobile LCP has **991 ms of headroom** to the 2.5 s threshold.

§02 Measured numbers

Lighthouse — median of 3 runs each

METRIC	MOBILE	DESKTOP
Performance score	100 (100/100/100)	100 (100/100/100)
First Contentful Paint	909 ms	248 ms
Largest Contentful Paint	1509 ms	368 ms
Speed Index	909 ms	248 ms
Time to Interactive	1509 ms	368 ms
Total Blocking Time	0 ms	0 ms
Cumulative Layout Shift	0.000	0.000
Max Potential FID	16 ms	16 ms
Main-thread work	0.7 s	—
JS bootup time	0.0 s	—

Run-to-run spread: mobile LCP 1504.8 / 1509.1 / 1512.7 ms; desktop LCP 365.3 / 368.0 / 374.4 ms.

Configuration recorded from the reports: Lighthouse 13.4.1, mobile emulation 412×823 @ DPR 1.75, desktop 1350×940 @ DPR 1, host `benchmarkIndex` ≈ 2850 (a fast machine — see the caveats section).

Real-browser, cold cache, unsimulated (CDP throttling, `browser-forensics.json`)

These are observed rather than modelled, and they cross-check the Lighthouse numbers.

SCENARIO	FCP	LCP	CLS	FONTS FINISH AT
Mobile, no throttle	108 ms	108 ms	0.00000	54 ms (all three)
Mobile, slow-4G + 4× CPU	708 ms	708 ms	0.00000	559 / 667 / 759 ms
Mobile, 400 kbps + 6× CPU	1884 ms	1884 ms	0.00000	1843 / 2283 / 2693 ms
Desktop, no throttle	112 ms	112 ms	0.00000	47–48 ms
Mobile, fonts blocked entirely	224 ms	224 ms	0.00000	never (all `error`)

LCP == FCP in every scenario. The largest element paints in the very first frame — there is no second-wave LCP candidate. That is the healthiest possible shape for this metric.

Interaction latency (INP proxy — measured, 4× CPU throttle)

Scripted: all six native `<details>` FAQ items toggled, plus a CTA hover+click. 22 `event` timing entries captured.

- Worst interaction duration: **72 ms** (threshold for "good" INP is 200 ms)
- Worst *processing* time: **2.4 ms** — the remainder is presentation/frame delay, not script
- CLS before interaction: 0.00000 · after all interactions: 0.00000

This is lab interaction latency, not field INP. See "what I could not measure".

§03 Resource inventory (measured)

	VALUE
Requests	8
Transferred	95,636 B (93.4 KiB)
Decoded	170,760 B (166.8 KiB)
Scripts	0 requests, 0 bytes
Third-party	0 requests, 0 bytes

By type:

TYPE	REQUESTS	TRANSFERRED	% OF TRANSFER
Font	3	75,528 B	79.0%
Document (HTML)	1	9,700 B	10.1%
Stylesheet	2	8,889 B	9.3%
Image (SVG)	1	886 B	0.9%
Other (favicon SVG)	1	633 B	0.7%
Script	0	0 B	0%
Media	0	0 B	0%

Per-request detail (transfer / decoded):

```
/                              9,700 /  52,880   text/html
http/1.1
/fonts/JetBrainsMono-latin.woff2 40,800 /  40,480   font/woff2
/fonts/SpaceGrotesk-latin.woff2  22,640 /  22,320   font/woff2
/fonts/Orbitron-latin.woff2      12,088 /  11,768   font/woff2
/_astro/index.af0tTXyW.css        8,013 /  39,481   text/css
/logo.svg                           886 /   1,180   image/svg+xml
/fonts.css                          876 /   2,285   text/css
/favicon.svg                        633 /     366   image/svg+xml
```

The near-zero-JS claim is confirmed, not inferred. `find dist -name '*.js'` returns **0 files**; `dist/_astro/` contains exactly one file and it is the CSS bundle. The only `<script>` in the HTML is a single `application/ld+json` block, which is data, not executable. Lighthouse reports script bootup time 0.0 s and TBT 0 ms. The FAQ is 6 native `<details>` elements with no JS behind them.

FlowForge

DRAWING TITLE

Resource inventory (measured)

SHEET	DWG NO.	REV	STATUS
05 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

§04 Self-contained claim: CONFIRMED — zero requests leave the origin

Verified two independent ways:

1. Lighthouse `network-requests` audit: 8 requests, all `http://localhost:4321/...`. Filtering for anything not on that origin returns **0**. The `third-party` row of `resource-summary` reads **0 requests / 0 bytes**.
2. Puppeteer request interception across all five scenarios, capturing every request the browser emitted: **EXTERNAL=0** in each.

There are no `preconnect` / `dns-prefetch` hints to external origins in the HTML, no external CSS `@import`, no analytics, no font CDN. The three `rel=preload` hints are all same-origin font paths.

§05 Findings, ranked by real-world impact

F1 — MEDIUM · The 40 KB mono font is preloaded but has *no above-fold consumer on mobile*

Measurement. `JetBrainsMono-latin.woff2` is 40,800 B transferred — **42.7% of the entire page weight** and the single largest asset. It is preloaded at high priority in `<head>`. I enumerated every element intersecting the first viewport and its computed font-family:

- **Mobile (412×823): above-fold text uses `Space Grotesk` (5 elements) and `Orbitron` (3 elements). JetBrains Mono users above the fold: NONE.**
- Desktop (1350×940): 38 above-fold elements use JetBrains Mono (the hero terminal frame).

The hero terminal figure sits at `top: 827.1px` on mobile against an 823 px viewport — it begins roughly 4 px *below* the fold. On desktop it starts at `top: 727.7px` inside a 940 px viewport, so desktop genuinely needs it.

Why it matters. On mobile the browser is told to fetch 40 KB at highest priority, in parallel with — and competing against — the render-blocking CSS and the two fonts that *are* needed for the LCP element. In the 400 kbps scenario this is visible in the arrival order: Orbitron 1843 ms, Space Grotesk 2283 ms, JetBrains Mono 2693 ms; the mono file occupies the pipe for ~410 ms after the fonts that actually gate the hero.

Recommendation. Make the mono preload viewport-conditional, keeping desktop's benefit and removing mobile's contention:

```
<link rel="preload" href="/fonts/JetBrainsMono-latin.woff2"
as="font"
      type="font/woff2" crossorigin media="(min-width: 768px)">
```

The `@font-face` rule stays as-is, so mobile still gets the font — just fetched at normal priority when the terminal frame scrolls into view, rather than racing the hero. Expected effect is on LCP under constrained bandwidth; it will not move the localhost numbers, which is precisely why it is worth doing before a real deployment rather than after.

F2 — MEDIUM · Two render-blocking stylesheets, one of which is an 876-byte round trip

Measurement. Lighthouse `render-blocking-insight` **scores 0** on both form factors — the only failing audit on the page.

BLOCKING RESOURCE	TRANSFER	MOBILE WASTED	DESKTOP WASTED
`/_astro/index.af0tTXyW.css`	8,013 B	305 ms	84 ms
`/fonts.css`	876 B	155 ms	—
Est. total savings		**290 ms**	**60 ms**

`fonts.css` contains nothing but five `@font-face` declarations. It costs a full render-blocking request to deliver 484 B (brotli) of content that the browser needs before it can resolve any font.

Recommendation, in priority order.

1. **Inline the `@font-face` block into `<head>` and delete the separate `fonts.css` request.** It is 2,285 B raw / 484 B compressed — smaller than the HTTP overhead of fetching it. This removes one render-blocking round trip outright (155 ms modelled on mobile).
2. **Consider inlining the main CSS too.** At 39,481 B raw / 6,882 B brotli it fits comfortably inside the initial congestion window. This is a single-page site, so the usual argument for keeping CSS as a separately-cacheable file is weak — there is no second page to amortise it across. The trade-off is that a repeat visitor re-downloads the CSS with the HTML instead of getting a 304; given `index.html` must be served `no-cache` anyway, that cost is small and the first-visit win is 305 ms modelled on mobile.

If you keep the CSS external, at minimum merge `fonts.css` into the Astro bundle so there is one blocking stylesheet rather than two.

F3 — MEDIUM (deploy) · gzip only, no brotli; and no real cache headers

Measurement. Every text response came back `Content-Encoding: gzip`. I compressed each asset both ways:

FILE	RAW	GZIP -9	BROTLI -Q11	BROTLI SAVES
`index.html`	52,880	9,345	7,527	19.5%
`_astro/index.af0tTXyW.css`	39,481	7,683	6,882	10.5%
`fonts.css`	2,285	575	484	15.9%
`logo.svg`	1,180	579	496	14.4%
`flowforge-logo.svg`	4,522	1,287	1,100	14.6%
`favicon.svg`	366	256	207	19.2%
Total	**100,714**	**19,725**	**16,696**	**15.4% (3,029 B)**

Brotli would remove ~3 KB from the compressible payload — about 15% of the non-font bytes. On the HTML alone (the one asset on the critical path before anything else can be discovered) it saves 1,818 B.

Every response also carried `Cache-Control: no-cache`. **That is an Astro preview-server artifact, not a defect in the build** — but it means the production cache policy is currently unspecified and must be set at deploy time (see the caching section).

The server correctly did **not** apply `Content-Encoding` to the `.woff2` files. That is right: woff2 is already Brotli-compressed internally, and re-compressing it wastes CPU for nothing. Preserve that behaviour in production.

F4 — LOW · A non-composited animation drives layout on the main thread

Measurement. Lighthouse `non-composited-animations` flags `div.z-scanner-track__pip` running the `z-scanner-sweep` keyframe animation, with `failureReason: "Unsupported CSS Property: left"`.

Animating `left` cannot be handed to the compositor, so every frame costs layout + paint on the main thread. It passes today (score 1, TBT 0 ms) because the element is 74×2 px and there is nothing else competing — but it is a needless main-thread cost on a page that otherwise has none, and it will be the first thing to jank on a low-end device.

Recommendation. Animate `transform: translateX()` instead of `left`, and add `will-change: transform`. This is a pure win with no visual difference. The CSS already respects reduced motion (durations collapse to `1ms` under a `prefers-reduced-motion` media query), so that path is unaffected.

F5 — LOW (deploy) · Font files and `fonts.css` carry no content hash

Measurement. Of the 14 files in `dist/`, exactly one is content-hashed:

```
HASHED    _astro/index.af0tTXyW.css
UNHASHED  fonts.css, index.html, favicon.svg, logo.svg, logo-mark.svg,
          logo-mono-black.svg, logo-mono-white.svg, flowforge-logo.svg,
          fonts/{Orbitron,SpaceGrotesk,JetBrainsMono}-*.woff2   (5 files)
```

Only the hashed file can safely take `immutable`, year-long caching. The five font files are the longest-lived, highest-value cache entries on the site (75 KB of the 93 KB payload) and they are exactly the ones that cannot be given an aggressive policy without risking a stale-font incident if a subset is ever regenerated.

Recommendation. Either move the fonts into the hashed asset pipeline, or accept the caching policy in the table below and treat any future font regeneration as requiring a manual CDN purge. Note that F2's recommendation to inline `@font-face` also removes `fonts.css` from this list.

F6 — LOW (hygiene) · Three unreferenced SVGs ship in the build

Measurement. Grepping `index.html` for each SVG: `logo.svg` ×2, `favicon.svg` ×1, `flowforge-logo.svg` ×1, and `logo-mark.svg`, `logo-mono-white.svg`, `logo-mono-black.svg` **×0 each** — 2,676 B of never-referenced files. They are never fetched, so the **runtime cost is exactly zero**; this is deploy-artifact hygiene, not a performance issue. Worth a note only so nobody assumes they are load-bearing.

F7 — INFO · CSS is 74.1% used at initial load

Measurement. Chrome CSS coverage at load: **30,962 of 41,756 B used (74.1%)**, leaving 10,794 B unused *at initial render*. Lighthouse's own `unused-css-rules` audit reports no waste worth flagging.

Most of that 10,794 B is genuinely used further down a 14,446 px page (this is a long single-page site), so it is deferred use rather than dead code. Compressed, the unused portion is well under 2 KB. **No action recommended** — splitting critical CSS here would cost more in complexity and an extra request than it returns.

One measurement note for anyone re-running this: coverage reports `fonts.css` as `usedAtLoad=0`. That is an instrumentation artifact — coverage tracks style *rules* that match elements, and a stylesheet containing only `@font-face` descriptors has none. The file is not unused; three of its five faces are actively loaded.

F8 — POSITIVE · `unicode-range` splitting works exactly as intended

Verified on the wire rather than read off the CSS. In all five browser scenarios and all six Lighthouse runs, **exactly 3 font requests** were made — the three `latin` subsets. `SpaceGrotesk-latin-ext.woff2` (18,924 B) and

`JetBrainsMono-latin-ext.woff2` (15,204 B) were **never fetched**. `document.fonts` confirms it from the other side: three faces `loaded`, and the two latin-ext faces sitting at status `unloaded`.

A Latin-only visitor therefore never pays for the 34,128 B of latin-ext. Those files are correctly-declared future capacity, not dead weight on the critical path.

---

§06 Font strategy audit

FILE	BYTES	PRELOADED	FETCHED	NEEDED ABOVE FOLD (MOBILE)	NEEDED ABOVE FOLD (DESKTOP)
`Orbitron-latin.woff2`	11,768	yes	yes	**yes** — `h1`, top 165 px	**yes**
`SpaceGrotesk-latin.woff2`	22,320	yes	yes	**yes** — LCP element, top 289 px	**yes**
`JetBrainsMono-latin.woff2`	40,480	yes	yes	**no** (see F1)	**yes** — 38 elements
`SpaceGrotesk-latin-ext.woff2`	18,924	no	**no**	n/a	n/a
`JetBrainsMono-latin-ext.woff2`	15,204	no	**no**	n/a	n/a

Verdict: the strategy is sound, with one mobile-specific correction (F1).

- The right files are preloaded on desktop — all three preloads have above-fold consumers.
- No font is downloaded and left unused: all three fetched faces reach `status: "loaded"` and all three are applied to real elements.
- `unicode-range` splitting is correct and verified on the wire (F8).
- Fonts are same-origin, `crossorigin` is present on the preloads (required for fonts even same-origin — correct), and `font-display: swap` is set on all five faces.

One structural gap worth recording: none of the `@font-face` rules declare `size-adjust`, `ascent-override`, `descent-override` or `line-gap-override`, and there is no metrics-matched fallback face. (The `size-adjust:100%` that appears in the CSS is `-webkit-text-size-adjust` on `html` — a different, unrelated property.) Normally that is the classic `font-display: swap` CLS trap. **Here it does not fire**, and I verified that rather than assuming it — see the CLS section. It remains a latent risk if the hero copy, its `font-size`, or the fallback stack changes.

FlowForge

DRAWING TITLE

Font strategy audit

SHEET	DWG NO.	REV	STATUS
08 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

§07 CLS forensics

Measured CLS: 0.00000. Zero layout-shift entries recorded, in every scenario.

I did not rely on the default Lighthouse run for this, because preloaded fonts arriving quickly on localhost would hide exactly the shift the brief is worried about. Five cold-cache runs with a `layout-shift` PerformanceObserver installed before navigation:

SCENARIO	FONTS ARRIVE	VS. FCP	SHIFT ENTRIES	CLS
Mobile, no throttle	54 ms	before	0	0.00000
Mobile, slow-4G + 4× CPU	559/667/759 ms	**759 ms vs FCP 708 ms — after**	0	0.00000
Mobile, 400 kbps + 6× CPU	1843/2283/2693 ms	**2283 & 2693 ms vs FCP 1884 ms — after**	0	0.00000
Desktop, no throttle	47–48 ms	before	0	0.00000
Mobile, fonts blocked	never	n/a	0	0.00000
Mobile, after 7 interactions	—	—	0	0.00000

In the 400 kbps run, Space Grotesk landed **399 ms after first paint** and JetBrains Mono **809 ms after** — a genuine swap window, on the font that renders the LCP element. **Still zero shift.**

Why it holds. The fonts-blocked run explains it. With every webfont failing to load, the hero geometry is byte-identical to the fully-loaded case:

```
                    fonts loaded                fonts blocked
h1          w=372   h=100.1   top=165     w=372   h=100.1   top=165
subhead     w=372   h=266.0   top=289.1   w=372   h=266.0   top=289.1
heroFig     w=372   h=487.9   top=827.1   w=372   h=487.9   top=827.1
```

Widths are container-driven and heights are driven by a fixed `line-height` ratio, so as long as the fallback wraps to the same number of lines, the box height cannot change — and the fallback stacks (`Inter` / `system-ui` for body, `Michroma` / `Arial Black` for display) wrap identically at these sizes. The swap changes glyphs, not geometry.

Two secondary confirmations: Lighthouse's `cls-culprits-insight` returns an empty item list on both form factors, and total document height differs by only 205 px between loaded and blocked (14,446 vs 14,241) — all of it below the fold, where it cannot contribute to CLS.

Dimensions on media boxes. Lighthouse `unsized-images` passes with zero items. Both `<img>` elements carry explicit intrinsic dimensions:

```
<img src="/logo.svg"           alt="Zerado"    width="364"
height="64">
<img src="/flowforge-logo.svg" alt="FlowForge" width="18"
height="18" loading="lazy">
```

There are no other replaced elements — no `<video>`, no `<iframe>`, no inline `<svg>`, and no raster images anywhere. The terminal frames and cover tiles are pure CSS, so they have no intrinsic-size race to lose; one `aspect-ratio: 3/4` is declared for the cover tiles. The below-the-fold `flowforge-logo.svg` is correctly `loading="lazy"` and, being 18×18 with explicit dimensions, reserves its space.

Caveat. CLS here is measured at load and across scripted `<details>` toggles. Expanding a FAQ item does move content below it, but those shifts are correctly excluded from CLS by `hadRecentInput` since they follow a user gesture — and I observed zero unattributed shifts after seven interactions.

---

§08 Render-blocking analysis

What blocks first paint: two linked stylesheets, and nothing else.

```
<link rel="stylesheet" href="/fonts.css">                    ←!— 876 B transfer —>
<link rel="stylesheet" href="/_astro/index.af0tTXyW.css">  ←!— 8,013 B transfer —>
```

The CSS is **linked, not inlined** — `<style>` tag count in `index.html` is **0**. There are no blocking scripts (there are no scripts at all), and no `@import` chains.

The network dependency tree is two levels deep and no deeper: document → {fonts.css, index CSS, 3 preloaded fonts}. The three fonts are discovered from the raw HTML via `rel=preload` rather than from the CSS, which is the right call — it means font fetching does not wait on the `fonts.css` round trip.

Is linked the right call at this size? For `fonts.css` — no. 484 B compressed does not justify a render-blocking round trip; inline it (F2). For the main bundle at 6,882 B brotli, linked is defensible but inlining is probably better on a single-page site (F2). Lighthouse puts the combined cost at **290 ms mobile / 60 ms desktop**.

Supporting measurements: LCP subpart breakdown is `timeToFirstByte 2.6 ms` + `elementRenderDelay 90.3 ms` on mobile (102.0 ms on desktop) — i.e. essentially *all* of observed LCP is render delay, and CSS is the only thing in that path. One 63 ms long task at 613 ms accounts for HTML parse + style recalc; main-thread work totals 0.7 s, with 0.0 s of it script.

`bf-cache` passes with zero failure reasons, so back/forward navigation restores instantly.

§09 Caching / deploy readiness

Recommended headers, given which files are content-hashed (F5):

PATH	CACHE-CONTROL	WHY
`/_astro/*`	`public, max-age=31536000, immutable`	Content-hashed (`index.af0tTXyW.css`); the filename changes when the bytes change
`/fonts/*.woff2`	`public, max-age=31536000, immutable` **only if** fonts are hash-named or frozen; otherwise `public, max-age=604800, stale-while-revalidate=86400`	**Not hashed** — a regenerated subset under the same name would be served stale for a year
`/fonts.css`	`public, max-age=3600, stale-while-revalidate=86400`	Not hashed, and it is the map to the font files. Best fixed by inlining it (F2) and deleting the file
`/index.html`	`no-cache` (or `max-age=0, must-revalidate`)	Must revalidate so a new deploy is picked up immediately; it references the hashed CSS
`/*.svg` **(logo, favicon)**	`public, max-age=604800`	Not hashed; a week is a safe compromise

Also required at deploy time:

- **Enable brotli** for `text/html`, `text/css`, `image/svg+xml`, `application/ld+json` (F3: 3,029 B / 15.4%). **Exclude** `font/woff2` — it is already Brotli-compressed internally and re-compressing burns CPU for no gain. The preview server got this right; preserve it.
- **Serve over HTTP/2 or HTTP/3.** The preview server is `http/1.1` (measured). With 8 requests this is a modest win, but the 3 preloaded fonts currently contend under HTTP/1.1 head-of-line rules; multiplexing removes that.
- **Set** `Vary: Accept-Encoding` on compressed responses so a CDN does not serve a brotli body to a gzip-only client.
- Ship `Content-Type: font/woff2` on the font files (already correct here).
- The build emits no `_headers` / `_redirects` / `netlify.toml` / `vercel.json` and no `Cache-Control` policy of its own — **every header above must be configured on the host or CDN.** Nothing in `dist/` will do it for you.

Nothing in the output would actively hurt on a CDN: no query-string-keyed assets, no cookies set, no redirects (`document-latency-insight` confirms `noRedirects: true`), and no origin-dependent content.

FlowForge	DRAWING TITLE	SHEET	DWG NO.	REV	STATUS
	Caching / deploy readiness	11 OF 14	FP-LP-PERF-01	A · 2026-08-25	● IN REVIEW

§10 What would change on a real deployment

This is the part a localhost Lighthouse score cannot tell you, so let me be explicit about which way each factor pushes.

Worse in production:

1. **TTFB is the big one.** Measured origin response: **2 ms**. A real CDN edge hit is typically tens of milliseconds and an origin miss can be far more — I did not measure this and will not invent a figure. The useful framing is arithmetic on what I *did* measure: since LCP here is `TTFB + render delay` and TTFB is currently ~3 ms, **mobile LCP ≈ 1509 ms + (real TTFB − 3 ms)**, holding everything else equal. With 991 ms of headroom to the 2.5 s threshold, production TTFB would have to exceed roughly one second before LCP fails. That is a wide margin, but it is the margin being spent.
2. **TLS and connection setup.** Localhost has no TLS handshake and no DNS lookup. A cold HTTPS connection adds a DNS resolution plus a TLS handshake before the first byte — real cost on a first visit, ~0 on repeat visits with connection reuse.
3. **Real devices are slower than a throttled fast Mac.** Lighthouse recorded `benchmarkIndex ≈ 2850`. A 4× CPU slowdown on this machine is still meaningfully faster than a genuine mid-tier Android. The page's defence is that it has *no JavaScript at all* — TBT is 0 ms and bootup is 0.0 s, so there is almost nothing for a slow CPU to be slow at. This is where the near-zero-JS architecture earns its keep, and it is why I am comfortable with the PASS.
4. **Real networks are lossy and variable.** Simulated throttling models bandwidth and latency but not packet loss, congestion, or radio wake-up. The 400 kbps + 6× CPU run is the closest thing I have to a worst case, and it produced FCP/LCP 1884 ms with CLS 0 — still inside the LCP budget.

Better in production:

5. **Brotli** (F3) cuts the HTML on the critical path by 1,818 B.
6. **HTTP/2 or HTTP/3** removes the HTTP/1.1 contention between the three preloaded fonts.
7. **CDN edge caching** means most visitors never touch the origin, and repeat visitors with the recommended headers fetch **0 bytes of font and CSS** — leaving just a revalidated `index.html`.

Net expectation. LCP will rise by roughly the real TTFB and fall slightly from brotli and HTTP/2. CLS should stay 0.000 — it is structural, held up by fixed `line-height` and explicit image dimensions rather than by network timing, and I verified it survives even total font failure. TBT/INP should stay at zero because

there is no JavaScript to block on. **The risk that this page fails Core Web Vitals in production is low, and the dominant variable is hosting TTFB, not anything in the build.**

The one change I would make before deploying is F1 (viewport-conditional mono preload), precisely because it is invisible on localhost and only pays off on a constrained real connection.

FlowForge

DRAWING TITLE
What would change on a real deployment

SHEET	DWG NO.	REV	STATUS
12 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

§11 What I could NOT measure, and why

1. **Field INP.** INP is a field metric — the p75 of real users' interaction latencies. I measured *lab* interaction latency with scripted clicks (worst 72 ms at 4× CPU throttle, worst processing 2.4 ms) and Lighthouse's `max-potential-FID` (16 ms) and TBT (0 ms). These are strong proxies and all sit far inside "good", but none of them *is* INP. Only RUM from real traffic can produce that number.
2. **Real TTFB / CDN behaviour.** The page is not deployed. Everything I measured came from an Astro preview server on localhost at 2 ms. I have deliberately not estimated a production TTFB figure; the sensitivity relationship above is arithmetic on measured values, not a prediction.
3. **Production cache-header behaviour.** All responses returned `Cache-Control: no-cache` from the preview server. Repeat-visit performance, revalidation cost, and CDN hit ratio are therefore unmeasured — they depend entirely on host configuration that does not exist yet.
4. **Brotli in situ.** I measured brotli savings by compressing the built files directly (`brotli -q11` vs `gzip -9`). The server only ever served gzip, so the *transfer* numbers in this report are all gzip numbers. The brotli column is a real measurement of the files, not of a response.
5. **Cross-browser vitals.** All measurements are Chromium. LCP, CLS and INP are Chromium-only APIs; Safari and Firefox do not report them. Cross-browser rendering is the `qa` lane's scope.
6. **HTTP/2 and HTTP/3 effects.** The preview server is HTTP/1.1 only, so I could not measure multiplexing gains — I could only observe the HTTP/1.1 contention it causes.
7. **Long-session / memory behaviour.** With zero JavaScript there is no heap to leak and no listener to strand, so I ran no soak test. Worth stating plainly: this was a judgement not to measure, not a measurement.
8. **Anything under `site/`.** Read-only lane, per scope — no code was modified and nothing was rebuilt. Every recommendation above is a recommendation, not an applied change.

Measured, not asserted. The numbers are the verdict.

FlowForge
ENGINEERED ARTIFACT

DRAWING TITLE

Zerado landing page — Core Web Vitals & performance report

DISCIPLINE	SHEET	DWG NO.	REV	STATUS
PERFORMANCE	14 OF 14	FP-LP-PERF-01	A · 2026-08-25	IN REVIEW

LEGEND — Field — blueprint · Ink — annotation · Accent — registration · Pass — verdict

Rendered from performance-report.md

powered by **FlowForge**